ShelfSense AI - Phase 2 MVP Progress Report

Course:	**MAE301**
Team:	**Outlier 5**
Team members:	**Gael Barba, Joseph Franco, Kai Jones, Trent Patton**

https://81a11873600d3ddc57.gradio.live

1. Objective and current MVP definition

Our Phase 1 proposal defined ShelfSense as an event-aware grocery demand forecasting system for store managers and inventory planners. The target MVP was: a manager selects a store and product family, and the system returns a short-horizon sales forecast plus a traffic alert that explains whether holidays or local events are contributing to the spike.

For Phase 2, we implemented that idea as a working forecasting pipeline with these deliverables:

1. a reproducible retail-style dataset generator,
2. a store-level customer traffic model,
3. a store + product-family sales model,
4. an ablation study comparing event-aware vs non-event-aware forecasting,
5. a 14-day recursive inference pipeline (the first 7 days are the core manager-facing horizon), and
6. a lightweight Gradio demo for selecting a store and family and viewing the forecast.

2. Main technical risk and bottleneck

The biggest technical risk from Phase 1 was whether calendar / event features actually add useful signal beyond ordinary time-series lags. A second risk was whether we could produce a manager-facing explanation without pretending the model is more causal than it really is.

To address those risks, Phase 2 focused on three concrete questions:

- Does an event-aware model beat a simple baseline?

- Does it especially help on event-heavy days?
- Can we convert model output into a short operational summary that a grocery manager could use?

3. What has been built so far

3.1 Data pipeline

We built a reproducible Arizona grocery scenario dataset shaped around the MVP assumptions so we could validate the modeling and product flow end-to-end before full public-dataset ingestion in Phase 3. Instead of one generic random event stream, the data now uses store-specific city context for Tempe, Phoenix, Mesa, and Scottsdale, with grocery-relevant holidays, city-specific local event weekends, weather seasonality, payday effects, and promotions.

Current synthetic dataset characteristics:

- 4 stores
- 6 product families (produce, beverages, snacks, dairy, bakery, frozen)
- 731 daily timestamps from 2024-01-01 to 2025-12-31
- 17,544 sales rows and 2,924 store-traffic rows
- External drivers: promotions, holidays, city-specific local events, weekend flags, payday flags, and an Arizona-style heat/temperature proxy

This lets us test the full event-aware idea while keeping the code fast and fully reproducible, while also staying much closer to the Phase 1 product story of combining grocery demand with outside public signals.

3.2 Forecasting models

We implemented two forecasting targets:

- Transactions model: predicts daily store-level transaction count as a proxy for customer influx.
- Sales model: predicts daily unit sales for each (store, product family) pair.

For both targets we compared three approaches:

1. Seasonal naive baseline using a 7-day lag.
2. XGBoost without event features.
3. XGBoost with event-aware features.

3.3 Feature engineering

We engineered lag and rolling-window features plus exogenous drivers:

- Lag 1 / 7 / 14 day demand
- 7-day and 28-day rolling means
- Weekday, week-of-year, month, quarter
- Promotion flag
- Holiday flag + holiday name
- Local event flag + event type
- Payday flag
- Weekend flag
- Heatwave / temperature proxy
- Predicted transaction count used as a store-traffic signal for family-level sales

3.4 Recursive forecasting and explanation layer

We built a recursive inference script that:

1. predicts future store traffic,
2. injects that traffic forecast into the sales model,
3. generates product-family forecasts for the next horizon, and
4. produces a short explanation such as:

Expect above-normal beverages demand because of festival, promotion activity.

This explanation layer is intentionally simple and rule-based on top of model outputs; it is meant as a manager-facing summary, not a causal proof engine.

4. Technical approach

4.1 Train / test protocol

We used a time-based split rather than a random split.

- Training set: all data except the final 56 days
- Test set: final 56 days of the historical timeline

This better matches the real deployment scenario where the model only sees the past and must generalize to future dates.

4.2 Model choice

We selected XGBoost regressors for the MVP because they are a strong fit for medium-sized structured tabular forecasting problems with lag features and mixed calendar inputs. They are also easier to iterate on quickly than a heavier sequence model.

4.3 Why this is aligned with the proposed MVP

The Phase 1 proposal argued that simple heuristics can miss interactions between weekday, promotions, seasonality, holidays, and local events. The Phase 2 experiments were designed specifically to test that claim by comparing a lag-only / no-event setup against an event-aware setup.

5. Evidence of progress

5.1 Quantitative results

Key result	Value
Sales WMAPE improvement: event-aware vs seasonal baseline	61.9%
Sales WMAPE improvement: event-aware vs no-event XGBoost	15.3%
Sales event-day MAE reduction vs baseline	70.8%
Traffic WMAPE improvement: event-aware vs seasonal baseline	46.5%
Traffic WMAPE improvement: event-aware vs no-event XGBoost	25.1%
Traffic event-day MAE reduction vs baseline	52.7%

Full benchmark table:

Model Name	target	mae	rmse	wmape	Event day mae	Non event day mae

seasonal_naive_7	transactions	74.4598	96.775	0.0735	145.938	62.5469
xgboost_no_events	transactions	53.218	76.7803	0.0525	144.78	37.9577
xgboost_event_aware	transactions	39.8338	51.8104	0.0393	69.0121	34.9707
seasonal_naive_7	sales	36.6417	56.4219	0.1941	77.9542	29.7563
xgboost_no_events	sales	16.4888	22.7714	0.0873	33.7613	13.6101
xgboost_event_aware	sales	13.9648	18.1817	0.074	22.7471	12.501

Interpretation:

- The event-aware model is the best model for both targets.
- The gain is especially strong on event days, which is exactly the core product hypothesis.
- The no-event XGBoost model already beats the naive baseline, but adding event/context features gives another measurable jump.

5.2 Error analysis

Two observations stood out:

1. Event-heavy days are where the baseline fails most. The seasonal baseline has much larger MAE on holiday / event days than on ordinary days.
2. Event-aware features reduce spike miss errors. The full model is noticeably better at following sharp changes around promotions, holidays, and local events.

5.3 Qualitative forecast examples

Sample 7-day forecast for Store S002 / beverages:

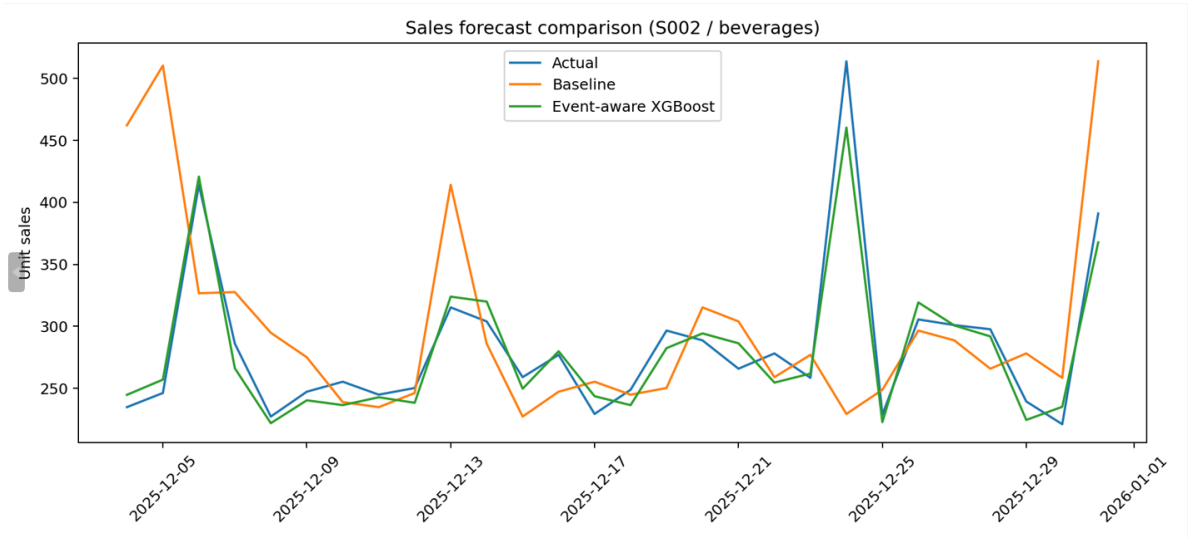
Date	Pred sales	Baseline sales	Uplift pct	Pred transactions	Holiday name	Local event name	On promotion
2026-01-01	339.8	273.9	24.1	1109	New Years day	none	0

2026-01-02	314.6	347.5	-9.5	1252	none	none	0
2026-01-03	387.2	340.6	13.7	1283	none	none	1
2026-01-04	305.9	368.7	-17	1223	none	none	0
2026-01-05	247	285.4	-13.4	1040	none	none	0
2026-01-06	408.8	295.9	38.2	1213	none	festival	1
2026-01-07	326.8	418.6	-21.9	1179	none	none	1

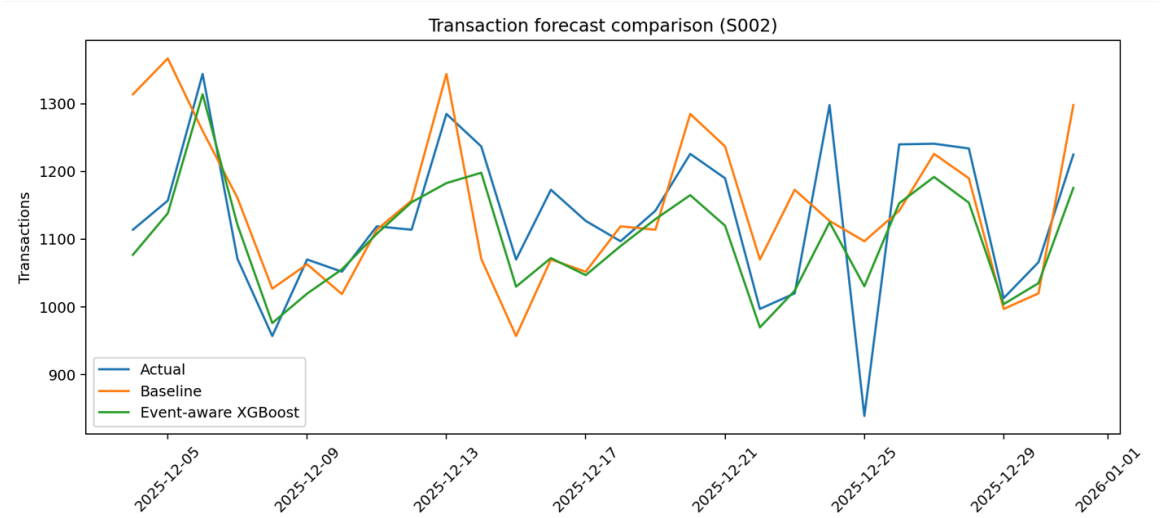
This sample is useful because it shows both positive uplift days (for example Jan 1 and Jan 6) and normal / below-baseline days, which is more realistic than only showing spikes.

5.4 Visual artifacts

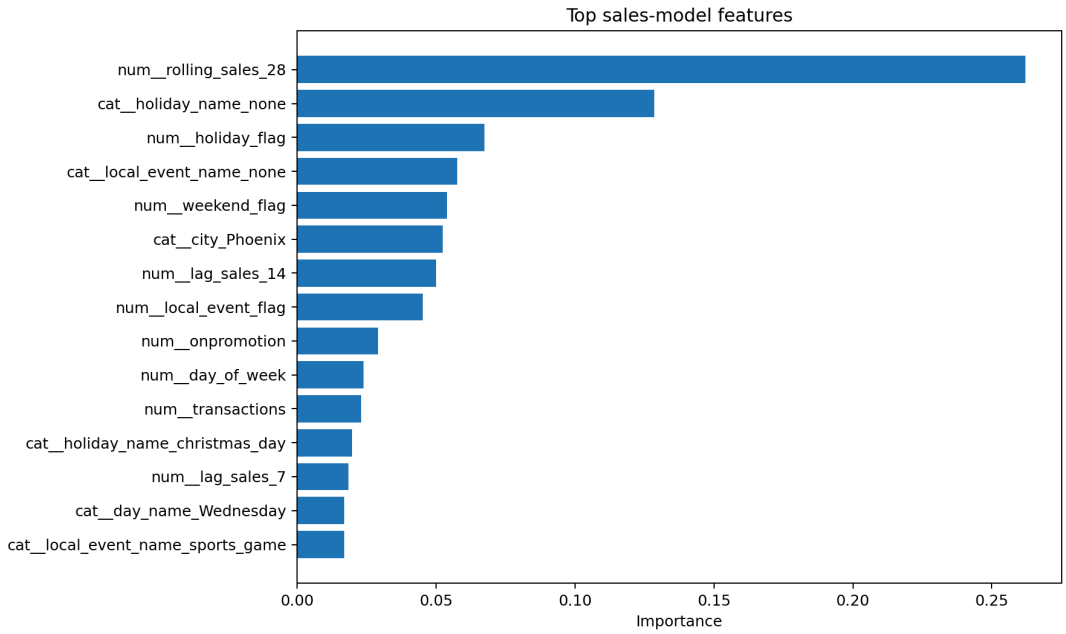
Sales-model comparison



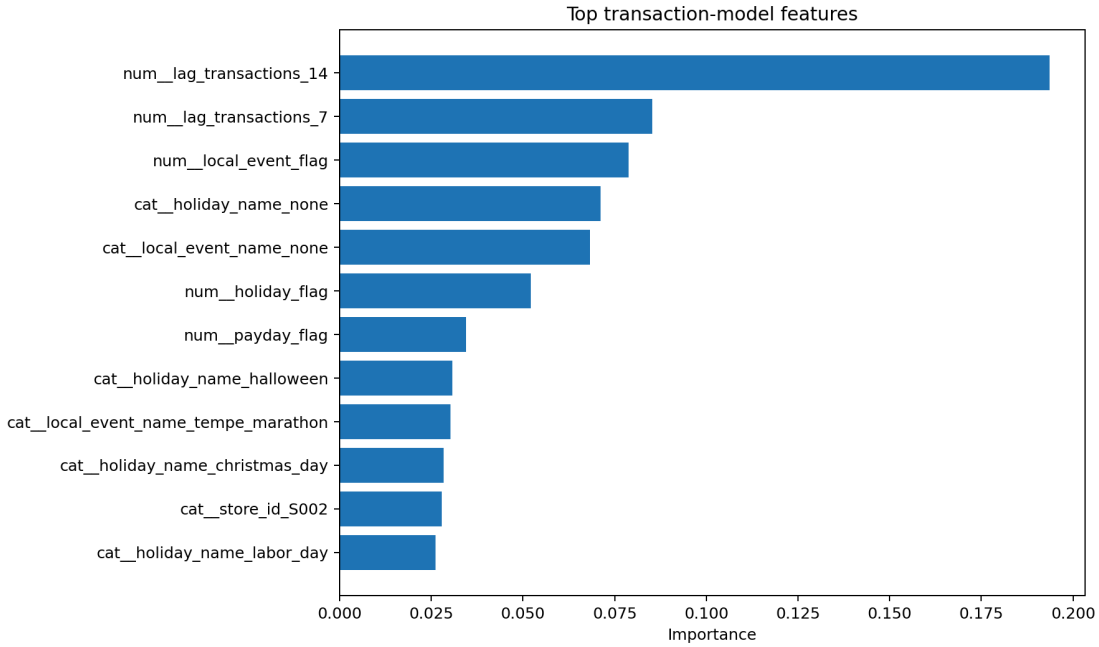
Transaction-model comparison



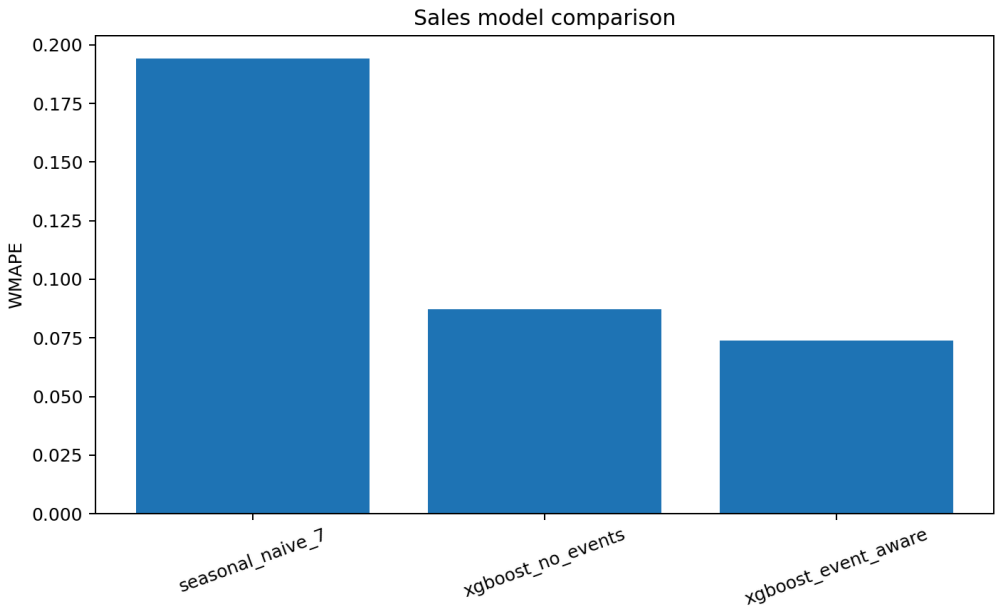
Sales feature importance



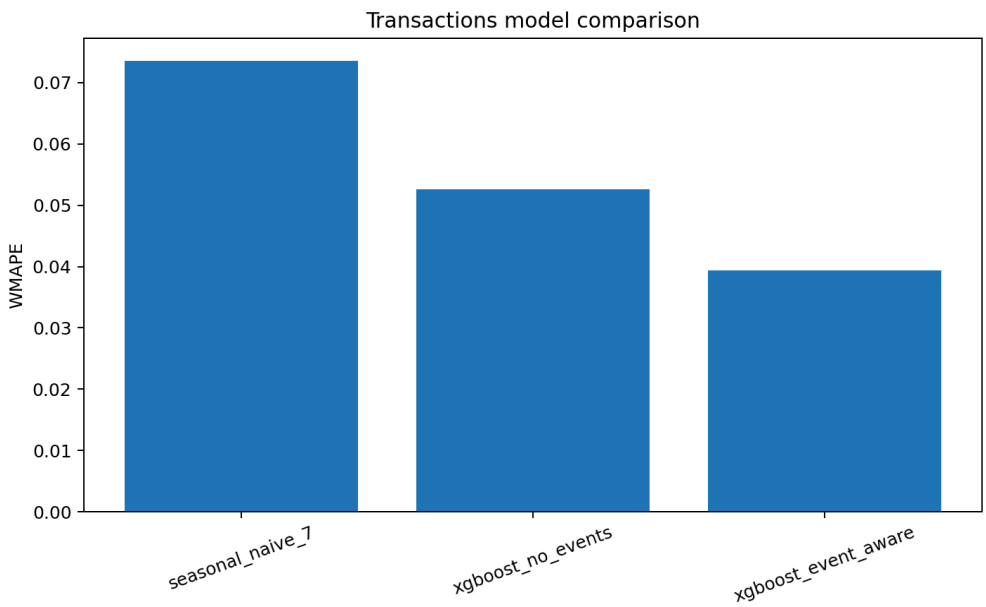
Transaction feature importance



WMAPE comparison (sales)



WMAPE comparison (transactions)



5.5 What the feature importance says

Top sales-model features:

Feature	Importance
num__rolling_sales_28	0.3942
num__weekend_flag	0.0643
cat__holiday_name_none	0.0587
cat__local_event_name_none	0.0483
num__local_event_flag	0.0462
num__holiday_flag	0.0458
num__onpromotion	0.0421
num__day_of_week	0.0345

Top transactions-model features:

Feature	Importance
num__lag_transactions_14	0.316
num__lag_transactions_7	0.1059
cat__store_id_S002	0.0565
num__local_event_flag	0.0412
cat__holiday_name_halloween	0.0381

num__holiday_flag	0.0354
num__day_of_week	0.0342
num__rolling_transactions_28	0.0324

The feature ranking is consistent with the product story: recent demand history matters most, but weekend / holiday / local-event / promotion signals are materially important and help the model react to demand spikes.

6. Current MVP demo

6.1 User flow

The current demo supports this workflow:

1. The manager chooses a store.
2. The manager chooses a product family.
3. System returns short-horizon sales forecast.
4. System returns a store-traffic forecast.
5. System highlights likely spike drivers in plain language.

6.2 System architecture

```
flowchart LR
    A[Calendar + promotions + event inputs] --> B[Transaction forecasting model]
    H[Historical sales and traffic] --> B
    A --> C[Sales forecasting model]
    H --> C
    B --> C
    C --> D[7 to 14 day family-level forecast]
    D --> E[Manager-facing explanation layer]
    E --> F[Demo UI / dashboard]
```

7. What works, what does not yet work, and what remains for Phase 3

7.1 What already works

- End-to-end training pipeline
- Baseline vs ablation benchmarking
- Store-level traffic forecasting
- Product-family sales forecasting
- Recursive future forecasting
- Manager-facing explanation strings
- Reproducible demo flow through demo_app.py

7.2 What does not yet work well enough

- We are not yet using the real Favorita / M5 raw data in the current Phase 2 pipeline.
- Local event data is simulated rather than pulled from a live source.
- Explanations are helpful operationally but are not causal explanations.
- The model forecasts product families, not full SKU-level assortment.
- We have not yet enforced inventory/business constraints such as nonnegative stock caps, perishability penalties, or ordering rules.

7.3 Lessons from failed / weaker iterations

- A lag-only baseline is too brittle around event spikes.
- A stronger learner without event inputs still misses too much signal on high-variance days.
- Recursive forecasting needs careful handling to avoid leakage and to ensure future traffic predictions feed downstream sales predictions correctly.
- During implementation we had to refactor grouped rolling-window features so they were computed correctly for each time series rather than accidentally misaligned across groups.

8. Current limitations and open risks

1. Real-data ingestion is still the biggest open item. The architecture is ready, but the current evidence is on a representative synthetic benchmark.
2. Event quality matters. A production version will depend on the coverage and timeliness of future event sources.
3. Granularity tradeoff remains real. Family-level forecasting is tractable now; SKU-level forecasting will require more careful hierarchy handling.

4. Explanation trust must be handled carefully. The explanation layer should be presented as an operational summary, not as a proof of causality.

9. Plan for Phase 3

Before Phase 3, the team should prioritize these next steps:

1. Replace the synthetic benchmark with an initial real-data pipeline using the planned public grocery dataset.
2. Keep the same model benchmark structure so results are directly comparable.
3. Integrate a real holiday/event source into the calendar builder.
4. Improve the UI so a manager can filter by store and product family and export the 7-day plan.
5. Add a clearer operational output such as recommended extra units to stock relative to recent average demand.
6. Expand evaluation with family-level slices and event-day case studies.

10. Reproducibility and repository contents

Main files in /phase2/:

- src/generate_synthetic_data.py - reproducible retail-style data generator
- src/train_models.py - feature engineering, training, evaluation, plots
- src/make_forecast.py - recursive future forecast generation
- demo_app.py - lightweight Gradio interface
- artifacts/ - plots, metrics, forecast CSVs, and summary markdown
- README.md - setup and run instructions

11. Conclusion

Phase 2 demonstrates substantial technical progress toward the ShelfSense MVP:

- The forecasting core exists
- The event-aware hypothesis is supported by the benchmark
- A manager-facing forecast flow is implemented
- The remaining work for Phase 3 is clear and bounded

The strongest takeaway from Phase 2 is that event-aware features improved both sales and traffic forecasting, especially on the exact high-r